Ponte BLE–TCP/IP com handoff entre dois PCs (Polar Verity Sense)

Oct 19, 2026 · @Rafael Cardoso

Restrições do BLE que definem o desenho

A ideia faz sentido e é um padrão conhecido ("BLE roaming" com coordenador distribuído), mas quatro restrições do BLE moldam o projeto:

1. **Um periférico BLE aceita uma única conexão central por vez.** Enquanto o Verity Sense está conectado ao PC A, ele deixa de anunciar (advertising) e o PC B não consegue vê-lo, ou seja, o B não mede o RSSI da pulseira enquanto ela está com o A.
2. **O RSSI de uma conexão ativa só é visível para quem está conectado.** O A lê o RSSI da conexão; o B fica cego.
3. **O Verity Sense pode aceitar mais de um cliente**, segundo a documentação da Polar, mas isso varia por firmware e por tipo de stream (PPG, ACC, HR). Teste no seu hardware antes de apostar nisso: se aceitar duas conexões simultâneas, o handoff fica quase sem perda.
4. **Conectar leva de centenas de ms a alguns segundos** (scan + connect + descoberta de serviços + início dos streams). Sem dupla conexão, o realista é minimizar o buraco nos dados, não eliminá-lo.

Como o nó B "vê" a pulseira

Como o B não enxerga a pulseira enquanto ela está com o A, há três estratégias; a recomendação é testar a A primeiro e, se o hardware não permitir, usar a C com histerese.

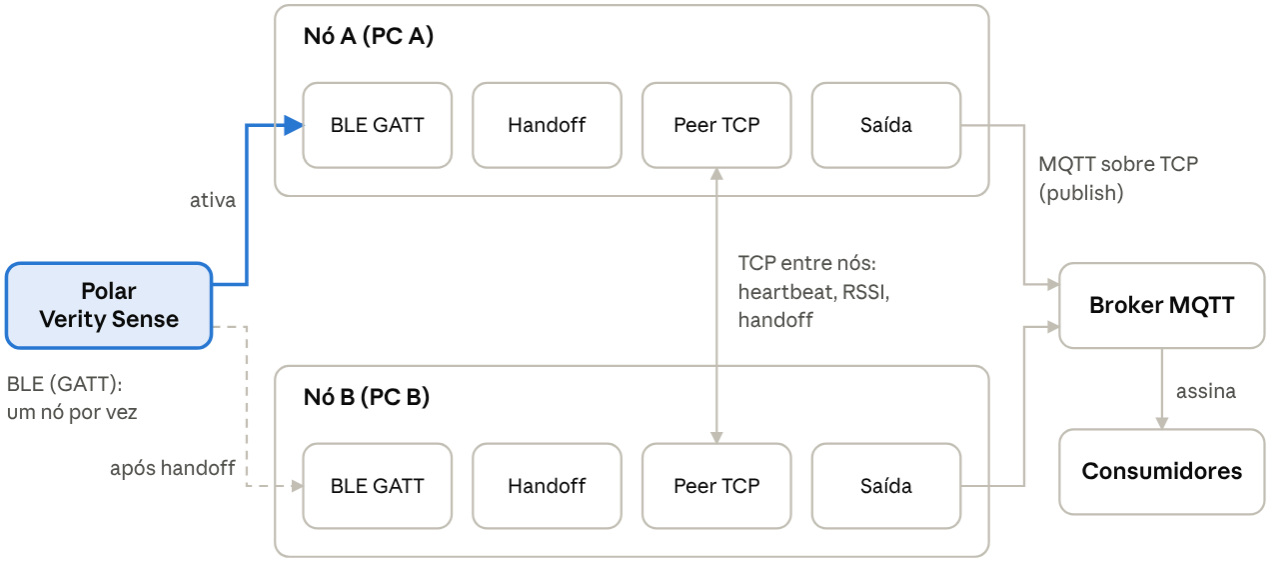
Estratégia	Como funciona	Vantagem	Custo / risco
A. Dupla conexão (ideal)	Ambos ficam conectados e comparam o RSSI de conexão. Quem tem melhor sinal vira o "ativo" (repassa dados ao consumidor); o outro fica em standby conectado.	Handoff é só trocar o papel, sem reconectar.	Depende de a pulseira aceitar duas conexões simultâneas.

Estratégia	Como funciona	Vantagem	Custo / risco
B. Sondagem por desconexão controlada	O A se desconecta periodicamente (ou quando o RSSI cai abaixo do limiar) para a pulseira voltar a anunciar, e o B faz scan e mede.	Funciona com uma conexão por vez.	Intrusivo: gera lacunas nos dados.
C. Degradação do A + tentativa do B	Quando o RSSI do A cai abaixo do limiar, o A avisa o B ("estou ficando fraco"). O A solta a pulseira e o B tenta conectar, com retorno ao A se o B falhar.	Não exige dupla conexão nem sondagem periódica.	Há um buraco nos dados durante a troca; precisa de fallback e histerese.

Arquitetura do sistema

Cada PC roda o mesmo serviço: o BLE liga a pulseira ao nó ativo, os nós conversam por TCP e a saída de dados publica num broker MQTT que os consumidores assinam.

Dois nós com o mesmo serviço: BLE na ponta, TCP entre eles, MQTT na saída



arquitetura · pulseira, 2 nós, broker MQTT

A linha contínua é a conexão BLE ativa e a tracejada é o caminho que passa a valer após o handoff; como a pulseira conversa com um nó por vez, apenas o nó ativo tem dados para publicar.

Fluxo do software

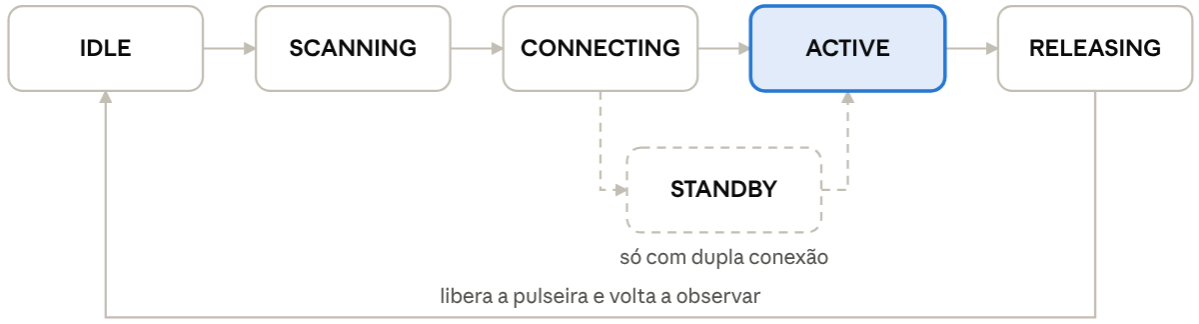
Cada nó roda o mesmo serviço, composto por quatro partes, e segue uma máquina de estados com sete passos principais de handoff.

Componentes de cada nó

- **Camada BLE (GATT client):** scan, connect, subscribe e leitura de RSSI. Em Python, o `bleak` cobre bem; a leitura de RSSI de conexão ativa varia por plataforma (mais acessível no BlueZ/Linux, existe `readRSSI` no CoreBluetooth/macOS, limitada no Windows).
- **Camada de peer (TCP):** conexão entre os dois serviços, com heartbeat, troca de RSSI e mensagens de controle.
- **Máquina de estados do handoff.**
- **Saída de dados:** o stream de HR/PPG/ACC repassado ao consumidor por um ponto único (por exemplo, um broker ou socket que o consumidor assina), independentemente de qual nó está ativo.

Estados por nó: `IDLE` → `SCANNING` → `CONNECTING` → `ACTIVE` (conectado e transmitindo) → `RELEASING` → `IDLE`. Com dupla conexão, acrescenta-se `STANDBY`.

Só o nó em ACTIVE transmite; STANDBY só existe com dupla conexão



estados de cada nó · 5 estados + STANDBY opcional

O nó percorre IDLE, SCANNING, CONNECTING e ACTIVE; ao soltar a pulseira passa por RELEASING e volta a IDLE, e o STANDBY entra entre CONNECTING e ACTIVE somente quando há dupla conexão.

Fluxo principal

1. **Descoberta de peers:** os dois serviços se encontram (mDNS ou IP fixo) e mantêm heartbeat TCP. Se o peer cair, o nó trabalha sozinho (degradação graciosa).
2. **Eleição inicial:** ambos escaneiam; quem vê a pulseira com melhor RSSI conecta primeiro e o outro fica em `IDLE`, observando.
3. **Monitoramento contínuo (nó ativo):** lê o RSSI da conexão a cada ~500 ms a 1 s, aplica filtro (média móvel ou EMA, pois o RSSI é muito ruidoso) e publica o valor filtrado ao peer.
4. **Gatilho com histerese:** inicia o handoff só quando o RSSI filtrado do ativo fica abaixo de um limiar (ex.: -80 dBm) e o candidato está melhor por uma margem (ex.: +10 dB) durante N segundos, o que evita o "ping-pong" quando a pessoa fica no meio.
5. **Handshake de transferência:** o ativo envia `HANDOFF_REQUEST` (com timestamp e último estado dos streams) e o peer aceita se puder conectar.
   - **Make-before-break (com dupla conexão):** o peer conecta, confirma que os dados fluem e só então o antigo envia `RELEASE` e se desconecta.
   - **Break-before-make (sem dupla conexão):** o ativo se desconecta, envia `RELEASED` e o peer conecta imediatamente, já com a configuração dos streams pronta (UUIDs e comandos de início de medição) para encurtar o buraco.
6. **Fallback:** se o peer não conectar dentro de um timeout (ex.: 5 s), ele avisa e o nó original tenta reconectar; sem isso, a pulseira fica órfã.
7. **Cooldown:** após um handoff, bloqueie novos handoffs por alguns segundos.

Detalhes que costumam morder

Seis pontos de implementação costumam causar problemas em campo:

- **Reconexão rápida:** guarde o endereço/ID do dispositivo e use conexão direta em vez de scan completo. No macOS o identificador é um UUID por host (não o MAC), então cada nó guarda o seu.
- **Estado dos streams:** o Verity Sense precisa de comandos de início de streaming (protocolo PMD da Polar) a cada nova conexão. Encapsule isso numa rotina de "bring-up" reutilizável.
- **Relógio entre nós:** sincronize (NTP ou compensação) se os dados dos dois forem mesclados depois, para o timestamp não saltar no handoff.
- **Pareamento/bonding:** confira se a pulseira exige bonding e como cada SO guarda a chave.
- **Antena e adaptador:** a qualidade do adaptador BLE de cada PC afeta o RSSI. Calibre um offset por nó, senão um adaptador "enxerga" sistematicamente melhor que o outro.

- **Split brain:** se o TCP entre os nós cair no meio de um handoff, defina uma regra determinística (por exemplo, o nó com ID menor assume) para que não haja dois tentando conectar, nem nenhum.

Ordem sugerida de implementação

Comece pelo teste de dupla conexão (etapa 2), porque ele decide a arquitetura inteira.

1. **Prova de conceito:** um nó conecta, lê HR e RSSI de conexão com `bleak`.
2. **Teste de dupla conexão:** verificar se a pulseira aceita duas conexões simultâneas de dois PCs.
3. **Camada TCP:** heartbeat e troca de RSSI entre os nós.
4. **Máquina de estados:** handoff manual, disparado por comando.
5. **Handoff automático:** filtro de RSSI, limiar e histerese.
6. **Robustez e métricas:** fallback, cooldown e logs (duração do buraco de dados, número de handoffs, ping-pong).

Pergunta em aberto: qual sistema operacional roda em cada computador? Isso muda bastante o que dá para ler de RSSI de conexão ativa.